

非排序算法完整实现

```
冒泡排序：
void bubbleSort(vector<int& v>){
    size_t n=v.size(); bool swapped;
    for(size_t i=0;i<n-1;i++){
        swapped=false;
        for(size_t j=i;j<n-i;j++){
            if(v[j]>v[j+1]){swap(v[j],v[j+1]); swapped=true;}
            if(swapped) break; // 已有序
        }
    }
}

插入排序（埋障）：
void insertionSort(vector<int& v>){
    for(size_t i=1;i<v.size();i++){
        int key=v[i]; size_t j=i;
        while(j>0 && v[j-1]>key){v[j]=v[j-1]; --j;}
        v[j]=key;
    }
}

选择排序：
void selectionSort(vector<int& v>){
    for(size_t i=0;i<v.size();i++){
        size_t minIdx=i;
        for(size_t j=i+1;j<v.size();j++){
            if(v[j]<v[minIdx]) minIdx=j;
            if(minIdx!=i) swap(v[i],v[minIdx]);
        }
    }
}

稳定二级排序 comparator：
bool byBest(const Move&a,const Move&b){
    if(a.toRow!=b.toRow) return a.toRow<b.toRow; // 主键
    return a.toCol<b.toCol; // 次键
}
stable_sort(v.begin(),v.end(),byBest);
```

迭代器安全 & find_if

```
// 查找后必须检查！
auto it=find_if(v.begin(),v.end(),pred);
if(it!=v.end()){ // 安全使用 *it '/' // 不检查引用用end()-UB

// find_if 示例：查找落在第4行的move
bool landsOnRow4(const Move&m){return m.toRow==4;}
auto it=find_if(log.begin(),log.end(),landsOnRow4);
if(it!=log.end()){ cout<<"Found at "<<it-log.begin();
else cout<<"Not found";
```

防御性编程完整模式

```
bool undoLast(vector<Move& v> log){
    if(log.empty()){cout<<"Nothing to undo\n"; return false;}
    log.pop_back(); return true;

}

void printLastMove(const vector<Move& v> log){
    if(log.empty()){cout<<"No moves\n"; return;}
    cout<<"Last move to row "<<log.back().toRow<<endl;

}

bool isValidSquare(int row,int col){
    return row>=0 && row<BOARD_SIZE && col>=0 && col<BOARD_SIZE;

}

bool addMoveSafe(vector<Move& v> log, const Move& m){
    if(!isValidSquare(m.fromRow,m.fromCol){
        if(!isValidSquare(m.toRow,m.toCol)){return false;}
        log.push_back(m); return true;
    }
}

异常 vs if： 只用于正常状态(空log)，异常用于意外输入(用户输入索引)。
```

文件 I/O 完整示例

```
// 版本化存档
struct SaveGame{int level; vector<int> inventory; };
void save(const SaveGame& g, const char* path){
    ofstream out(path); out<<"v"<<'\n'; // 版本号
    out<<g.level<<'\n';
    out<<g.inventory.size()<<'\n'; // 长度前缀
    for(int id:g.inventory) out<<id<<'\n';

}

bool load(SaveGame& g, const char* path){
    ifstream in(path); if(!in) return false;
    int ver; in>>ver; if(ver!=1) return false;
    in>>g.level; size_t n; in>>n;
    g.inventory.resize(n); // 预分配
    for(size_t i=0;i<n;i++) in>>g.inventory[i];
    return static_cast<bool>(!in)

}

序列化规则： 永远别 write(&vec, sizeof(vec))； 逐字节序列化； 读写顺序一致。
```

二进制 POD struct 读写

```
struct PlayerPod{int id=0; float x=0.f; float y=0.f;}; // POD！
// 写入
ofstream out("p.bin",ios::binary);
out.write(reinterpret_cast<const char*>(&p), sizeof(p));

// 读取
ifstream in("p.bin",ios::binary);
in.read(reinterpret_cast<char*>(&q), sizeof(q));

仅基本类型 struct 可用（无 string/vector/虚函数/指针）。
```

MVC 完整示例结构

```
// Model (no raylib includes)
class GameModel {
public:
    int playerX, playerY, coinX, coinY, score;
    static constexpr int SPEED=5, WINDOW_W=999;
    int getPlayerX() const{return playerX;}
    void movePlayer(int dx,int dy){
        void update(); // collision + scoring
        void saveToFile(const char*) const;
    }

// View (const ref to Model)
class GameView {
public:
    void draw(const GameModel& m) const;

};

// Controller (handles input)
class GameController {
public:
    void handleInput(GameModel& m);

};

// main wires them together
```

菱形继承与虚基类完整示例

```
class Animal{public:string name; Animal(string n):name(n){};
class Flyer:virtual public Animal{ // ~ virtual!
public: int wingSpan; Flyer(string n,int w):Animal(n),wingSpan(w){}
};

class Swimmer:virtual public Animal{ // ~ virtual!
public: int swimSpeed; Swimmer(string n,int s):Animal(n),swimSpeed(s){}
};

class Duck:public Flyer,public Swimmer{public:
    Duck(string n,int w,int s):Animal(n) // 直接初始化虚基类！
        ,Flyer(n,w),Swimmer(n,s){}
};

Duck duck("Donald",50,10);
cout<<duck.name; // ✓ 无歧义！只有一份 Animal
```

头文件陷阱（期中易错）

```
// 双引号 "" vs 尖括号 <>
#include <ostream> <> // 尖括号：搜索系统标准路径
#include "myfile.h" // 双引号：先搜当前目录，再搜系统路径

// 重复包含问题：
// a.h 包含 b.h
// main.cpp 包含 a.h 和 b.h
// ~ 无保护则 b.h 被包含两次；报“重定义”错误

// 解决：头文件保护（include guard）
#ifndef MYFILE_H
#define MYFILE_H
// 头文件内容 ...
#endif

// 或：#pragma once（非标准但广泛支持）
```

对象数组及隐患

```
Professor faculty[3]; // 调用3次默认构造！
// 若 Professor 没有默认构造函数 ~ 编译错误
// 必须提供默认构造函数式初始化：
Professor faculty[3]{
    Professor(1981,"Dr.A"),
    Professor(1982,"Dr.B"),
    Professor(1983,"Dr.C");
};

// 数组析构：按逆序，faculty[2]先析构-faculty[0]最后
静态数组 vs 局部数组：全局数组静态存储期→程序全程；局部数组自动存储→离开作用域销毁。
```

指针数组与浅拷贝

```
Pawn* blackPawns[8]; Pawn* whitePawns[8];
for(int i=0;i<8;++i) blackPawns[i]=new Pawn(i);
// 错误！浅拷贝→两者指向同一对象
for(int i=0;i<8;++i) whitePawns[i]=blackPawns[i];
// 后果：whitePawns[0]→promote() 影响 blackPawns[0]
// 双循环 delete 时双量释放+UB/崩溃！

// 正确提供：深拷贝（各自 new）
for(int i=0;i<8;++i) whitePawns[i]=new Pawn(i);
```

深浅拷贝深入

编译器生成的默认拷贝是**逐成员拷贝**（member-wise copy）。对于基本类型（int/double 等），复制值。对于指针成员，只复制地址→两个对象指向同一内存。浅拷贝后果：两个对象共享同一堆内存，一个析构释放后另一个析构再释放→double free UB。一个修改也会影响另一个。深拷贝：分配新内存并复制值。判断是否需要深拷贝：类是否有指向动态内存的指针成员？有则三法则。

更多脑残陷阱

- short s=32768；→溢出（short范围-32768~32767）
- vector<string> v(5)；创建5个空string，不是5个“S”
- sizeof(string) 返回固定大小（内部指针+size+capacity），不等于字符串长度
- cout<<c.str(); 之前不能修改s（c.str）可能失效
- int p=malloc(sizeof(int)); → C++中不合法（需强转+不用C风格）
- int f(){return 1;} double f(){return 1.5;} → 不合法！仅返回值不同
- 类中 static 成员不占对象大小，static 成员函数不传 this
- 模板特化：template<> void swap<MyType>(MyType&a,MyType&b);
- for(auto x:v) vs for(auto& x=v); 前者拷贝，后者引用可修改
- auto x = expr; 推导去掉引用和顶层 const，auto& x = expr; 保留

构造函数隐式转换陷阱

```
class MyString{
public: MyString(const char*s){} // #explicit
};

void print(const MyString& s){
    print("hello"); // 隐式转换 const char*→MyString
}

// 若加 explicit：explicit MyString(const char*s){}
// print("hello"); // 编译错误！必须显式 print(MyString("hello"))
// 同理同理，MyString s = "hello"; // explicit 前缀

标准库中大部分单参数构造都是 explicit 的。
```

异常处理最佳实践

```
void replayMove(const vector<Move& v> log, int moveNumber){
    try{ const Move& m=log.at(moveNumber);
        cout<<"Move to row "<<m.toRow<<endl;
    } catch(const out_of_range& e){
        cout<<"Invalid! Log has "<<log.size()<<" moves.\n";
    }

}

// 捕获顺序：最具体的 exception 先，通用后
// 最后 catch(...) 兜底
// 为什么析构不能抛异常？对象锁定时再抛→std::terminate()
```

运算符重载注意事项

- operator+ 必须是成员函数（不能全局）
- operator+< operator+ 通常全局+友元（左操作数是 ostream/istream）
- 前置++x 返回引用（无拷贝），后置++x 返回拷贝（有拷贝参int）
- operator[] 通常提供 const 和非 const 两个版本
- operator+ 返回新对象（const 成员函数），operator+= 返回*this（非const）
- C++ 不允许重载的操作符：. . . *?: sizeof typeid

杂项补充

this 是 const 指针（不可重绑定），成员函数中隐式可用，sizeof(变量)=1 字节（C++保证每对象都有唯一地址），空基类优化(EBCO)：空基类可不占额外空间，alignof(T) 返回对齐要求，alignas(n) 指定对齐，decltype(expr) 获取表达式类型（不求值），static_assert(const,msg) 编译期断言，lambda: [capture](params)→ret(body)。[=] 值捕获全部，[&] 引用捕获全部。

期中高频错题专项：表达式合法性判断

1. int x=5,y=10; swap(x,y); ✓
2. int* p=&x; int& r=p; ✓ r 是 p 的引用
3. const int& r=42; ✓ const 引用可用绑字面量
4. int&& rr=42; ✓ 右值引用绑字面量
5. int& r=(a>b?a:b); × 三目返回右值，不能绑到普通引用
6. const int& r=(a>b?a:b); ✓ const 引用可绑
7. vector<int> v(10,5); ✓ 10 个元素都是 5
8. vector<int> v(10,5); ✓ 2 个元素 {10,5}
9. int a=5; void* p=&a; int* q=static cast<int*>(p); ✓
10. int x; cout<<x; × x 未初始化（局部变量值未定义）
11. static int x; cout<<x; ✓ 静态变量默认为 0
12. int a[5]={}; ✓ 全部初始化为 0
13. string s="hello"; s[0]='H'; ✓ mutable string
14. const string s="hello"; s[0]='H'; × const 不可修改
15. void f(int a[5]){cout<<sizeof(a);} 输出指针大小(8)，非数组大小(20)
16. char s[10]="ab"; s[11]='c'; ✓ 字符数组可修改
17. int a=1,b=2; int* p=&a; p=&b; ✓ 指针可重绑定
18. int a=1,b=2; int& r=a; r=b; ✓ r 重绑定？ NO！r=b 意思是 a=b！
19. Obj a; Obj b=a; vs Obj a; Obj b; b=a; 前者拷贝构造，后者拷贝赋值
20. int f(); int (*pf)()=f; ✓ 函数指针
21. Base* p=new Derived; delete p; 若无 virtual 析构→UB/泄漏
22. int a=1; int b=(a+++(a+a*2)); b=4（逗号表达式返回最后一项）
23. int a=1; cout<<(a+++(a+a*2)); 输出 3，a=2
24. if(int x=foo()){}/用x*/ ✓ C++17 if-with-init
25. for(int i=0;i<n;i++) int x=i; ✓ x 作用域只在循环体
26. int a=1,b=2; int& r=a; r=b; r 仍是 a 的引用，a 值变成 2
27. class A(); class B:private A(); class C:public B(); C 中无法访问 A 的 public 成员
28. int* p=new int[5]; delete p; 错！delete[]

C++ 版本特性速查

- C++11: auto, decltype, nullptr, 范围 for, lambda, constexpr, enum class, 智能指针, 初始化列表，-default=delete, override, final, 右值引用 &&
- C++14: 泛型 lambda, 返回类型推导
- C++17: if constexpr, inline namespace, 结构化绑定, string_view, optional
- 课程使用：C++17 编译 g++ -std=c++17

模板进阶

```
// 非类型模板参数
template<typename T, size_t N>
class Array{ T data[N]; public: size_t size() {return N;}};
Array<int,10> a; // 编译期固定大小

// 默认模板参数
template<typename T=int, size_t N=10>
class Buffer{ T buf[N]; };
Buffer<> b; // Buffer<int,10>

// 模板特化
template<typename T> class MyClass{ /*通用*/ };
template<> class MyClass<int>{ /*特化*/ };
// 特化：template<typename T> class MyClass<T>{ /*"指针特化"*/ };
模板定义放头文件（编译期实例化需要完整定义），每个实例化是独立类型。
```

static 成员详解

```
class Professor{
    static int totalProfessors; // 声明：所有对象共享
    string name;
public:
    Professor(string n):name(n){totalProfessors++;}
    ~Professor() {totalProfessors--;}
    static int getTotal(){return totalProfessors;} // 静态成员函数
    string getName()const{return name;} // 非静态需要this
};

int Professor::totalProfessors=0; // 必须在类外定义！（cpp文件中）
// 调用：Professor::getTotal(); // 无需对象
// 静态成员函数不能访问非静态成员（没有this）
```

this 指针详解

每个非静态成员函数隐含 this 指针指向调用对象。类型：T* const this（指针本身是常量，不可重绑定）。
class A{int x; public:
 A& setX(int x){ this->x=x; return *this; } // 链式调用
};
A a; a.setX(1).setX(2); // 链式调用
const 成员函数中 this 是 const T* const。

虚函数表 (vtable) 机制

每个含虚函数的类有一张 vtable（虚函数表），对象有一个 vptr（虚表指针）指向该表。调用 p->f() 时：通过 p 的 vptr 找到 vtable，查表得函数地址，调用。代价：一次间接寻址（几乎可忽略）。因此多态有微小运行时开销。

Rule of Three / Five

三：析构+拷贝构造+拷贝赋值 → 三个都需要五（C++11）：加 移动构造+移动赋值 零：能标标准库容器/智能指针就别自己管理资源

总结：考试常错点的

易错点	正解																																							
Student s();	Student s{};	是函数声明！应 Student s;		类定义缺；																																				
class Foo{};	分号必加		初始化顺序	声明序非列表序		static 成员定义	必须在类外 int A::x=0;		Base 析构非 virtual	多态 delete 泄漏		对象切片		用指针/引用传递		new[] 配 delete	必须 delete[]		v[] 越界	用 v.at(i) 安全		find 后不检查	必须 != end()		评分 ~ 比较	用 qps 比较	数组退化	传参丢失长度		v/6 编译通过	运行时错误		无符号死循环	i>=0 永真		默认参右往左	void f(int a=0,int b) 错		仅返回值重载	不合法